

The command `netstat -nat` lists all current TCP connections. How can I count the number of connections in each state?

A **socket** is an operating system–managed communication endpoint that allows processes to send and receive data. In Linux, sockets are maintained inside the kernel networking stack and are exposed to user-space processes as file descriptors. Any network service typically has at least one **listening socket** bound to an IP address and port, and for each accepted client connection, the kernel creates a separate connected socket.

Sockets are used because they provide a unified abstraction for communication between processes, both over the network and locally. Common socket types include **TCP stream sockets** for reliable, connection-oriented communication, **UDP datagram sockets** for connectionless communication, **UNIX domain sockets** for local inter-process communication, and **raw sockets** for low-level access to network packets.

The `netstat -nat` command displays a complete list of TCP sockets along with their current TCP state. Each line in the output represents one TCP socket. Instead of inspecting individual entries, the number of connections in each state can be determined by extracting the **State** column, grouping identical states, and counting how many times each state appears.

For example, a concise summary can be obtained with:

```
netstat -nat | awk 'NR>2 {print $6}' | sort | uniq -c
```

Common TCP states include **LISTEN**, which indicates that a service is waiting for incoming connections, and **ESTABLISHED**, which indicates that a full TCP connection is active and data can flow in both directions. Other states such as **SYN_SENT**, **SYN_RECV**, **FIN_WAIT1**, **FIN_WAIT2**, **CLOSE_WAIT**, **LAST_ACK**, **TIME_WAIT**, and **CLOSING** represent different stages of TCP connection setup and termination.

How can we read the `stdin`, `stdout`, and `stderr` streams of a running process using the `cat` command?

In Linux, `stdin`, `stdout`, and `stderr` are represented as file descriptors: 0 (`stdin`), 1 (`stdout`), and 2 (`stderr`). For a running process with PID `<PID>`, these streams can be accessed via `/proc/<PID>/fd/` and read using `cat` (subject to permissions), e.g. `cat /proc/<PID>/fd/1` for `stdout` or `cat /proc/<PID>/fd/2` for `stderr`.

To demonstrate redirecting **stderr** into a separate file, file descriptor redirection can be used. For example:

```
mohammad@template:~$ ls /path/that/does/not/exist 2> error.log
mohammad@template:~$ cat error.log
ls: cannot access '/path/that/does/not/exist': No such file or directory
```

In this example, the `ls` command produces an error message. By redirecting file descriptor 2 using ``2> error.log``, the `stderr` output is written to a separate file instead of being displayed on the terminal. The contents of the `stderr` stream are then verified using the ``cat`` command.

What is an inode and what is its purpose?

When an inode is allocated, the kernel creates the inode metadata and establishes a mapping between the file name and the inode by adding a directory entry. This mapping allows the file name to reference the inode within the filesystem.

When you run the `top` command, it shows process states such as running, sleeping, zombie, and stopped. Can you explain what each state means and when it occurs?

A **zombie process** is a process that has finished execution but still has an entry in the process table because its parent process has not yet collected its exit status using `wait()`. Zombie processes do not consume CPU or memory resources, but they remain visible until the parent handles them. While a zombie process represents a single process state, it can appear in different scenarios, such as short-lived zombies that are quickly cleaned up, or long-lived zombies caused by a parent process failing to properly reap its children.

The **load average**, as shown in tools like `htop`, represents the average number of processes that are either running on the CPU or waiting to be scheduled, including processes blocked in uninterruptible I/O. It is not a direct measure of CPU usage but rather an indicator of overall system load. On a multi-core system, load average values should be interpreted relative to the number of CPU cores.

In `htop`, memory usage is displayed using different colors. The **yellow memory area** represents cached and buffered memory used by the kernel to improve system performance, such as the page cache. This memory is not permanently allocated to applications and can be reclaimed quickly if needed. Therefore, a large yellow memory area does not indicate memory pressure but rather efficient use of available RAM.

If your server shows a high load average and a high number of context switches, what could this indicate?

A high load average combined with a high context switch rate usually indicates heavy contention and frequent scheduling activity, where the kernel spends significant time switching between runnable tasks instead of doing useful work. Common scenarios include:

1. **CPU-bound contention:** many processes are runnable at the same time and compete for limited CPU cores, increasing both run-queue pressure (load average) and scheduler time-slicing (context switches).
2. **Too many threads/processes:** an application with a large number of workers or threads can trigger excessive scheduling overhead, leading to a high context switch rate even if individual tasks do little work.
3. **I/O-bound workload with frequent blocking:** tasks repeatedly sleep and wake due to disk/network I/O or blocking system calls, which increases context switches and can keep load elevated depending on the type of waits.
4. **Memory pressure (paging/swapping):** insufficient RAM causes reclaim and/or swapping activity, slowing the system and increasing task wakeups and stalls, often reflected as high load and poor performance.
5. **Scheduling/priority effects:** misconfigured priorities or real-time tasks can increase preemption and switching, impacting overall responsiveness.

In such situations, a high load average together with frequent context switches usually indicates that tasks are spending a significant amount of time competing for CPU or other shared resources. Instead of making steady progress, processes are repeatedly preempted, blocked, and rescheduled by the kernel scheduler. As a result, scheduling overhead increases and the effective CPU time spent on useful work is reduced.

Overall, this pattern often suggests reduced throughput because the system is busy managing runnable tasks rather than executing productive CPU work.